1.PROBLEM STATEMENT:

1. PerformOperation isOdd(): Returns a lambda expression that checks if a number is odd (true) or even (false).
2. PerformOperation isPrime(): Returns a lambda expression that checks if a number is prime (true) or composite (false).
3. PerformOperation isPalindrome(): Returns a lambda expression that checks if a number is a palindrome (true) or not (false).

Input handling is provided by locked stub code.

- The first line contains an integer T (the number of test cases).
- The subsequent lines describe a test case in space-separated integers:
    - First integer: The condition to check (1 for Odd/Even, 2 for Prime, 3 for Palindrome).and then, Second integer: The number to be checked.

SOURCE CODE:

```
import java.io.*;
import java.util.*;
interface PerformOperation {
   boolean check(int a);
}
class MyMath {
   public static boolean checker(PerformOperation p, int num) {
      return p.check(num);
   }
   public PerformOperation isOdd() {
      return n -> n % 2 != 0;
   }
   public PerformOperation isPrime() {
      return n -> {
         if (n < 2) return false;
         for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) return false;
```

```
            }
            return true;
        };
    }
        public PerformOperation isPalindrome() {
        return n -> {
String s = Integer.toString(n);
            return s.equals(new StringBuilder(s).reverse().toString());
        };
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine());
        PerformOperation op;
        boolean ret = false;
        String ans = null;
        while (T-- > 0) {
            String s = br.readLine().trim();
            StringTokenizer st = new StringTokenizer(s);
            int ch = Integer.parseInt(st.nextToken());
            int num = Integer.parseInt(st.nextToken());
            if (ch == 1) {
                op = ob.isOdd();
                ret = MyMath.checker(op, num);
                ans = (ret) ? "ODD" : "EVEN";
            } else if (ch == 2) {
```

```
op = ob.isPrime();
                ret = MyMath.checker(op, num);
                ans = (ret) ? "PRIME" : "COMPOSITE";
            } else if (ch == 3) {
                op = ob.isPalindrome();
                ret = MyMath.checker(op, num);
                ans = (ret) ? "PALINDROME" : "NOT PALINDROME";
            }
            System.out.println(ans);
        }
    }
}
```

OUTPUT:

```
5
1 4
2 5
3 898
1 3
2 12
```

Your Output (stdout)

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

2.PROBLEM STATEMENT:

You are given a list of student information: **ID**, **FirstName**, and **CGPA**. Your task is to rearrange them according to their **CGPA** in decreasing order.

If two students have the same CGPA, arrange them according to their **FirstName** in alphabetical order. If those two students also have the same first name, order them according to their **ID** in ascending order. No two students have the same ID.

Input Format

- The first line of input contains an integer N, representing the total number of students.
- The next N lines contain the student information in the following format:

ID FirstName CGPA

Constraints

- $2 \leq N \leq 1000$
- $0 \leq ID \leq 100000$
- $5 \leq |FirstName| \leq 30$
- $0 \leq CGPA \leq 4.00$
- The name contains only lowercase English letters.
- The ID contains only integer numbers without leading zeros.
- The CGPA will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the rules, print the **FirstName** of each student on a separate line.

SOURCE CODE:

```
import java.util.*;

class Student {

    private int id;

    private String fname;

    private double cgpa;

    public Student(int id, String fname, double cgpa) {

        super();

        this.id = id;

        this.fname = fname;
```

```
        this.cgpa = cgpa;
    }
    public int getId() {
        return id;
    }

    public String getFname() {
        return fname;
    }

    public double getCgpa() {
        return cgpa;
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());
        List<Student> studentList = new ArrayList<Student>();
        while (testCases > 0) {
            int id = in.nextInt();
            String fname = in.next();
            double cgpa = in.nextDouble();
            Student st = new Student(id, fname, cgpa);
            studentList.add(st);
            testCases--;
        }
        // Sorting using a custom Comparator
        Collections.sort(studentList, new Comparator<Student>() {
```

```
            @Override
            public int compare(Student s1, Student s2) {
                if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
                    return Double.compare(s2.getCgpa(), s1.getCgpa()); // CGPA in
descending order
                } else if (!s1.getFname().equals(s2.getFname())) {
                    return s1.getFname().compareTo(s2.getFname()); // Name in
alphabetical order
                } else {
                    return Integer.compare(s1.getId(), s2.getId()); // ID in ascending order
                }
            }
        });
        for (Student st : studentList) {
            System.out.println(st.getFname());
        }
        in.close();
    }
}
```

OUTPUT:

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

Your Output (stdout)

```
Ashis
Fahim
Samara
Samiha
Rumpa
```

3.PROBLEM STATEMENT:

Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.

Return ***any array*** *that satisfies this condition*.

Example 1:

Input: nums = [3,1,2,4]

Output: [2,4,3,1]

Explanation: The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.

Example 2:

Input: nums = [0]

Output: [0]

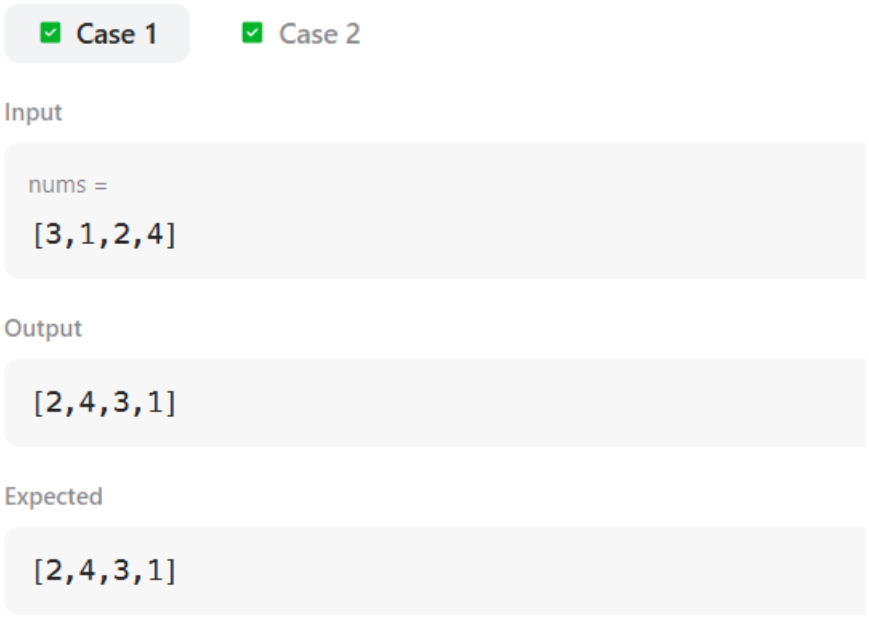
Constraints:

- 1 <= nums.length <= 5000
- 0 <= nums[i] <= 5000

SOURCE CODE:

```
import java.util.*;
class Solution {
    public int[] sortArrayByParity(int[] nums) {
        return Arrays.stream(nums)
                    .boxed()
                    .sorted((a,b)->Integer.compare(a%2,b%2))
                    .mapToInt(Integer::intValue)
                    .toArray();
    }
}
```

OUTPUT:



4.PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you will create a comparator and use it to sort an array.

The Player class is provided in the editor and contains two fields:

1. name: a string.
2. score: an integer.

Given an array of Player objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name.

To do this, create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing Comparator.compare(T o1, T o2).

Input Format

- The first line contains an integer, n, representing the number of players.
- Each of the next n lines contains a player's name and score (a string and an integer).

Constraints

- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing output to stdout. The locked stub code in Solution will instantiate a Checker object, use it to sort the Player array, and print each sorted element.

SOURCE CODE:

```
import java.util.*;

class Player {
    String name;
    int score;
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
```

```
}
class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            // Sort descending by score
            return b.score - a.score;
        } else {
            // Sort ascending by name if scores are equal
            return a.name.compareTo(b.name);
        }
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

OUTPUT:

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Your Output (stdout)

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

5.PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

Given an array of n Player objects, write a comparator that sorts them:

1. In order of decreasing score.
2. If two or more players have the same score, sort those players alphabetically by name.

To do this, you must create a Checker class that implements the Comparator<Player> interface and implements the int compare(Player a, Player b) method.

Input Format

Input from stdin is handled by locked stub code in the Solution class:

- The first line contains an integer, n, denoting the number of players.
- Each of the subsequent n lines contains a player's name and score, respectively.

Constraints

- $0 \leq$ score ≤ 1000
- Two players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing output to stdout. The locked stub code creates a Checker object, uses it to sort the Player array, and prints each sorted element.

SOURCE CODE:

```
Import java.util.*;

class Checker implements Comparator<Player> {
   @Override
   public int compare(Player a, Player b) {
      if (a.score == b.score) {
         return a.name.compareTo(b.name);
      }
```

```
        return Integer.compare(b.score, a.score);
    }
}
class Player{
    String name;
    int score;
    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}
class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

OUTPUT:

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Your Output (stdout)

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

6.PROBLEM STATEMENT:

You are given an m x n integer grid accounts where accounts[i][j] is the amount of money the ith customer has in the jth bank. Return *the wealth that the richest customer has.*

A customer's wealth is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: accounts = [[1,2,3],[3,2,1]]

Output: 6

Explanation:

1st customer has wealth = 1 + 2 + 3 = 6

2nd customer has wealth = 3 + 2 + 1 = 6

Both customers are considered the richest with a wealth of 6 each, so return 6.

SOURCE CODE:

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int rich = 0;
        for (int[] i : accounts) {
            int sum = 0;
            for (int money : i) {
                sum += money;
            }
            rich = Math.max(rich, sum);
        }
        return rich;
    }
}
```

OUTPUT:

Input

accounts =

```
[[1,2,3],[3,2,1]]
```

Output

```
6
```

Expected

```
6
```

7.PROBLEM STATEMENT:

Given a zero-based permutation nums (0-indexed), build an array ans of the same length where ans[i] = nums[nums[i]] for each 0 <= i < nums.length and return it.

A zero-based permutation nums is an array of distinct integers from 0 to nums.length - 1 (inclusive).

Example 1:

Input: nums = [0,2,1,5,3,4]

Output: [0,1,2,4,5,3]

Explanation: The array ans is built as follows:

ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]], nums[nums[5]]]

= [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]

= [0,1,2,4,5,3]

SOURCE CODE:

```
class Solution {


    public int[] buildArray(int[] nums) {
        int n = nums.length;


        for (int i = 0; i < n; ++i) {
            nums[i] += 1000 * (nums[nums[i]] % 1000);
        }
        for (int i = 0; i < n; ++i) {
            nums[i] /= 1000;
        }
        return nums;
    }
}
```

OUTPUT:

Input

nums =

```
[0,2,1,5,3,4]
```

Output

```
[0,1,2,4,5,3]
```

Expected

```
[0,1,2,4,5,3]
```

8.PROBLEM STATEMENT:

Given the array nums consisting of 2n elements in the form $[x_1,x_2,...,x_n,y_1,y_2,...,y_n]$.

Return the array in the form $[x_1,y_1,x_2,y_2,...,x_n,y_n]$.

Example 1:

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is [2,3,5,4,1,7].

Example 2:

Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]

SOURCE CODE:

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int len = nums.length;
        int[] ans = new int[len];
        for (int i = 0; i < len; i++) {
            if (i % 2 == 0) {
                // Even positions get elements from the first half
                ans[i] = nums[i / 2];
            } else {
                // Odd positions get elements from the second half (starting at index n)
                ans[i] = nums[n + (i / 2)];
            }
        }
        return ans;
    }
}
```

OUTPUT:

Input

nums =

```
[2,5,1,3,4,7]
```

n =

```
3
```

Output

```
[2,3,5,4,1,7]
```

Expected

```
[2,3,5,4,1,7]
```

9.PROBLEM STATEMENT:

Given an integer array nums sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in nums to be k. After removing duplicates, return the number of unique elements k.

The first k elements of nums should contain the unique numbers in **sorted order**. The remaining elements beyond index k - 1 can be ignored.

Custom Judge:

The judge will test your solution with the following code:

int[] nums = [...]; // Input array

int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;

for (int i = 0; i < k; i++) {

    assert nums[i] == expectedNums[i];

}

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: nums = [1,1,2]

Output: 2, nums = [1,2,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

SOURCE CODE:

```
class Solution {

    public int removeDuplicates(int[] nums) {

        if(nums.length == 0) return 0;

        int k = 1;
```

```
        for(int i = 1; i < nums.length; i++){
            if(nums[i] != nums[i-1]){
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }
}
```

OUTPUT:



10.PROBLEM STATEMENT:

There is a biker going on a road trip. The road trip consists of n + 1 points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array gain of length n where gain[i] is the **net gain in altitude** between points i and i + 1 for all (0 <= i < n). Return *the **highest altitude** of a point.*

Example 1:

Input: gain = [-5,1,5,0,-7]

Output: 1

Explanation: The altitudes are [0,-5,-4,1,1,-6]. The highest is 1.

SOURCE CODE:

```
class Solution {
  public int largestAltitude(int[] gain) {
    int currentAltitude = 0;
    // Highest altitude currently is 0.
    int highestPoint = currentAltitude;

    for (int altitudeGain : gain) {
      // Adding the gain in altitude to the current altitude.
      currentAltitude += altitudeGain;
      // Update the highest altitude.
      highestPoint = Math.max(highestPoint, currentAltitude);
    }

    return highestPoint;
  }
}
```

OUTPUT:

Input

gain =

[-5,1,5,0,-7]

Output

1

Expected

1

11.PROBLEM STATEMENT:

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4
- k is in the range [1, the number of unique elements in the array].
- It is **guaranteed** that the answer is **unique**.

SOURCE CODE:

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        for(int n : nums) {
            map.put(n, map.getOrDefault(n, 0) + 1);
        }
        List<Integer>[] freq = new ArrayList[nums.length + 1];
        for(int i = 0; i <= nums.length; i++) {
            freq[i] = new ArrayList<>();
        }

       for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            int frequency = entry.getValue();
            freq[frequency].add(entry.getKey());
```

```
        }
        int[] res = new int[k];
        int ind = 0;
        for(int i = freq.length - 1; i >= 0; i--) {
            for(int num : freq[i]){
                res[ind++] = num;
                if(ind == k) {
                    return res;
                }
            }
        }
        return new int[0];
    }
}
```

OUTPUT:

Input

nums =

[1,1,1,2,2,3]

k =

2

Output

[1,2]

Expected

[1,2]

12.PROBLEM STATEMENT:

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced *deck*) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

In this problem, you are given N integers. You need to find the **maximum number of unique integers** among all possible contiguous subarrays of size M.

Input Format

1. The first line of input contains two integers, N and M: representing the total number of integers and the size of the subarray, respectively.
2. The next line contains N space-separated integers.

Constraints

- $1 \leq N \leq 100{,}000$
- $1 \leq M \leq 100{,}000$
- $M \leq N$
- The numbers in the array range between 0 and $10{,}000{,}000$.

SOURCE CODE:

```
import java.util.*;

public class test {

    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        Deque<Integer> deque = new ArrayDeque<>();

        Set<Integer> set = new HashSet<>();

        int n = in.nextInt();

        int m = in.nextInt();

        int maxUnique = 0;

        for (int i = 0; i < n; i++) {

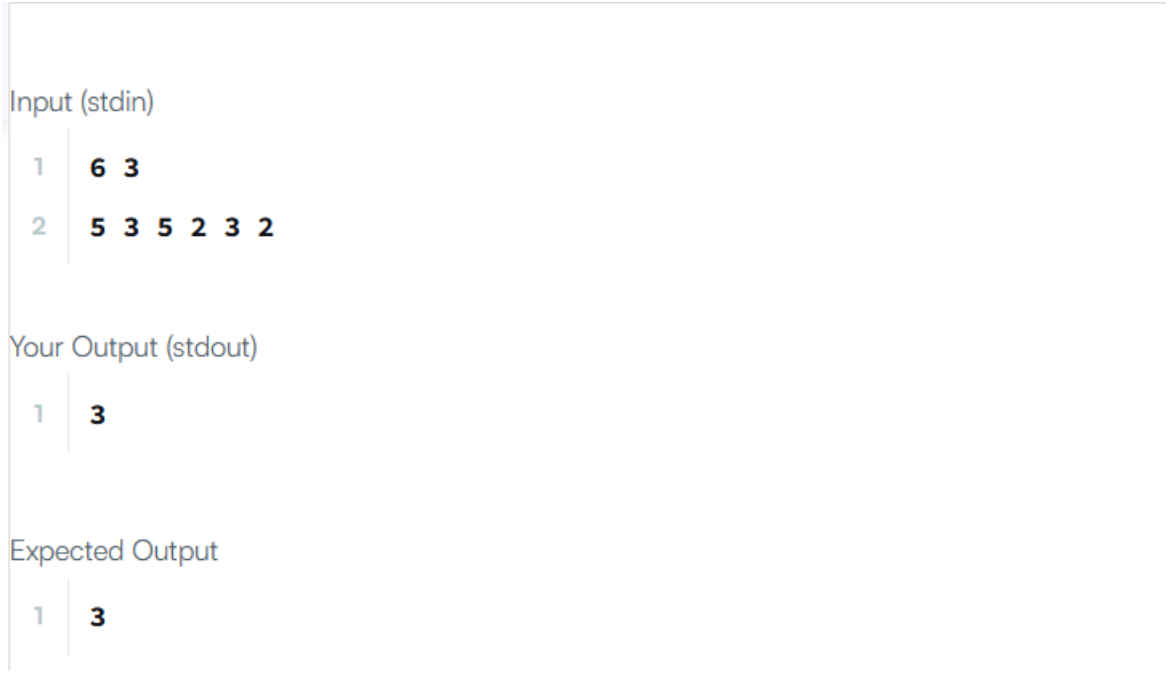
            int num = in.nextInt();

            deque.add(num);

            set.add(num);
```

```
            if (deque.size() == m) {
                if (set.size() > maxUnique) {
                    maxUnique = set.size();
                }
                if (maxUnique == m) {
                    break;
                }
                int first = deque.removeFirst();
                if (!deque.contains(first)) {
                    set.remove(first);
                }
            }
        }
        System.out.println(maxUnique);
    }
}
```

OUTPUT:



13.PROBLEM STATEMENT:

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values. $\{1,2,3\}$ is an example of a set, but $\{1,2,2\}$ is not a set. Today you will learn how to use sets in Java by solving this problem.

You are given T pairs of strings. Two pairs (a, b) and (c, d) are identical if $a = c$ and $b = d$. That also implies (a, b) is not the same as (b, a). After taking each pair as input, you need to print the number of unique pairs you currently have.

Input Format

- The first line contains an integer T denoting the number of pairs.
- Each of the next T lines contains two strings separated by a single space.

Constraints

- $1 \leq T \leq 100000$
- Length of each string is at most 5 and will consist of lowercase English letters only.

Output Format

Print T lines. In the ith line, print the number of unique pairs you have after taking the ith pair as input.

SOURCE CODE:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int t = s.nextInt();

        String [] pair_left = new String[t];

        String [] pair_right = new String[t];
```

```
        for (int i = 0; i < t; i++) {
            pair_left[i] = s.next();
            pair_right[i] = s.next();
        }
        HashSet<String> set = new HashSet<>();
        for (int i = 0; i < t; i++) {
            set.add(pair_left[i] + " " + pair_right[i]);
            System.out.println(set.size());
        }
        s.close();
    }
}
```

OUTPUT:

Input (stdin)

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Your Output (stdout)

```
1
2
2
```

14.PROBLEM STATEMENT:

The Calendar class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method findDay, which returns the day of the week for that date (in capital letters, e.g., SUNDAY, MONDAY, etc.).

Example

- Inputs: month = 8, day = 14, year = 2017
- Output: MONDAY

Function Description

Complete the findDay function in the editor.

findDay has the following parameters:

- int month: the month component of the date
- int day: the day component of the date
- int year: the year component of the date

Returns:

- string: the day of the week in capital letters (e.g., MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY).

Input Format

A single line of input containing the space-separated month, day, and year in MM DD YYYY format.

SOURCE CODE:

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.function.*;
```

```
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
class Result {
    public static String findDay(int month, int day, int year) {
        Calendar cal = Calendar.getInstance();
        cal.set(year, month - 1, day); // Calendar months are 0-indexed (0 = January)
        String[] days = {"SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY",
"THURSDAY", "FRIDAY", "SATURDAY"};
        return days[cal.get(Calendar.DAY_OF_WEEK) - 1];
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));


        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$",
"").split(" ");
        int month = Integer.parseInt(firstMultipleInput[0]);
        int day = Integer.parseInt(firstMultipleInput[1]);
        int year = Integer.parseInt(firstMultipleInput[2]);
        String res = Result.findDay(month, day, year);
        bufferedWriter.write(res);
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
```

```
    }
}
```

OUTPUT:

Input (stdin)

```
08 05 2015
```

Your Output (stdout)

```
WEDNESDAY
```

Expected Output

```
WEDNESDAY
```

15.PROBLEM STATEMENT:

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is YYYY-MM-DD as shown in the examples.

Examples

Example 1:

- Input: date1 = "2019-06-29", date2 = "2019-06-30"
- Output: 1

Example 2:

- Input: date1 = "2020-01-15", date2 = "2019-12-31"
- Output: 15

Constraints

- The given dates are valid dates between the years 1971 and 2100.

SOURCE CODE:

```
class Solution {
    public int daysBetweenDates(String date1, String date2) {
        return Math.abs(calcDaysFrom1971(date1) - calcDaysFrom1971(date2));
    }
    private int[] daysOfMonth = new int[]{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    private int calcDaysFrom1971(String date) {
        String[] dateArr = date.split("-");
        int y = Integer.parseInt(dateArr[0]), m = Integer.parseInt(dateArr[1]), d =
Integer.parseInt(dateArr[2]);
        for (int iy = 1971; iy < y; iy++) d += isLeap(iy) ? 366 : 365;
        for (int i = 0; i < m - 1; i++) d += daysOfMonth[i];
        if (m > 2 && isLeap(y)) d += 1; // on Feb leap year have 29 days instead of 28
        return d;
    }
    private boolean isLeap(int year) {
```

```
        return year % 400 == 0 || (year % 100 != 0 && year % 4 == 0);
    }
}
```

OUTPUT:

Input

```
date1 =
"2019-06-29"
```

```
date2 =
"2019-06-30"
```

Output

```
1
```

Expected

```
1
```

16.PROBLEM STATEMENT:

Given a string date representing a Gregorian calendar date formatted as YYYY-MM-DD, return *the day number of the year*.

Example 1:

Input: date = "2019-01-09"

Output: 9

Explanation: Given date is the 9th day of the year in 2019.

Constraints:

- date.length == 10
- date[4] == date[7] == '-', and all other date[i]'s are digits
- date represents a calendar date between Jan 1st, 1900 and Dec 31st, 2019**.**

SOURCE CODE:

```
class Solution {
    public int dayOfYear(String date) {
        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        int year = Integer.parseInt(date.substring(0, 4));
        int month = Integer.parseInt(date.substring(5, 7));
        int day = Integer.parseInt(date.substring(8, 10));
        if (isLeapYear(year)) {
            days[1] = 29;
        }
        int totalDays = 0;
        for(int i = 0; i < month - 1; i++){
            totalDays += days[i];
        }
        totalDays += day;
        return totalDays;
    }
```

```
    boolean isLeapYear(int year){
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    }
}
```

OUTPUT:



17.PROBLEM STATEMENT:

Given a date, return the corresponding day of the week for that date.

The input is given as three integers representing
the day, month and year respectively.

Return the answer as one of the following values {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"}.

Note: January 1, 1971 was a Friday.

Example 1:

Input: day = 31, month = 8, year = 2019

Output: "Saturday"

Example 2:

Input: day = 18, month = 7, year = 1999

Output: "Sunday"

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

SOURCE CODE:

```
class Solution {
    public String dayOfTheWeek(int day, int month, int year) {
        String[] daY =
{"Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"};
        int[] daysOfMonth = {0,31,28,31,30,31,30,31,31,30,31,30,31};
        if( leapYear(year) ){
            daysOfMonth[2] = 29;
        }
        int daysOfYear = 4;
        for( int i = 1971 ; i < year ; i++){
            if( leapYear(i)){
                daysOfYear += 366;
            }else{
```

```
            daysOfYear += 365;
        }
    }
    for( int i = 1 ; i < month ; i++){
        daysOfYear += daysOfMonth[i];
    }
    daysOfYear += day;
    return daY[daysOfYear % 7];
  }
  private boolean leapYear(int year){
    if((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        return true;
    return false;
  }
}
```

OUTPUT:

Input

day =

31

month =

8

year =

2019

Output

"Saturday"

Expected

"Saturday"

18.PROBLEM STATEMENT:

There are a number of students in a school who wait to be served. Two types of events, **ENTER** and **SERVED**, can take place:

- **ENTER**: A student with some priority enters the queue to be served.
- **SERVED**: The student with the highest priority is served (removed) from the queue.

A unique ID is assigned to each student entering the queue. The queue serves the students based on the following **priority criteria**:

1. The student having the highest **Cumulative Grade Point Average (CGPA)** is served first.
2. Any students having the same CGPA will be served by **name in ascending case-sensitive alphabetical order**.
3. Any students having the same CGPA and name will be served in **ascending order of the [1][2] ID**.

Implementation Details

Create the following two classes:

1. **Student** class:
    - Constructor: Student(int id, String name, double cgpa)
    - int getID(): returns the ID of the student.
    - String getName(): returns the name of the student.
    - double getCGPA(): returns the CGPA of the student.
2. **Priorities** class:
    - Implement the method: List<Student> getStudents(List<String> events) to process all the given events and return all the students yet to be served in priority order.

SOURCE CODE:

```
import java.util.ArrayList;

import java.util.Comparator;

import java.util.List;

import java.util.PriorityQueue;

import java.util.Scanner;

class Student {
```

```
    private final int id;
    private final String name;
    private final double cgpa;
    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    public int getID() {
        return id;
    }
    public String getName() {
        return name;
    }
    public double getCGPA() {
        return cgpa;
    }
}
class Priorities {
    public List<Student> getStudents(List<String> events) {
        PriorityQueue<Student> queue = new PriorityQueue<>(
            Comparator.comparing(Student::getCGPA).reversed()
                .thenComparing(Student::getName)
                .thenComparing(Student::getID));
        for (String event : events) {
            String[] parts = event.split(" ");
            if (parts[0].equals("ENTER")) {
                String name = parts[1];
                double cgpa = Double.parseDouble(parts[2]);
```

```
            int id = Integer.parseInt(parts[3]);
            queue.add(new Student(id, name, cgpa));
        } else if (parts[0].equals("SERVED")) {
            queue.poll();
        }
    }
    List<Student> remainingStudents = new ArrayList<>();
    while (!queue.isEmpty()) {
        remainingStudents.add(queue.poll());
    }
    return remainingStudents;
  }
}
public class Solution {
  private final static Scanner scan = new Scanner(System.in);
  private final static Priorities priorities = new Priorities();
  public static void main(String[] args) {
    int totalEvents = Integer.parseInt(scan.nextLine());
    List<String> events = new ArrayList<>();
    while (totalEvents-- != 0) {
        String event = scan.nextLine();
        events.add(event);
    }
    List<Student> students = priorities.getStudents(events);
    if (students.isEmpty()) {
        System.out.println("EMPTY");
    } else {
        for (Student st: students) {
            System.out.println(st.getName());
```

```
        }
      }
    }
}
```

OUTPUT:

nput (stdin)

```
12
ENTER John 3.75 50
ENTER Mark 3.8 24
ENTER Shafaet 3.7 35
SERVED
SERVED
ENTER Samiha 3.85 36
SERVED
ENTER Ashley 3.9 42
ENTER Maria 3.6 46
ENTER Anik 3.95 49
ENTER Dan 3.95 50
```

Your Output (stdout)

```
Dan
Ashley
Shafaet
Maria
```

Expected Output

```
Dan
Ashley
Shafaet
Maria
```

19.PROBLEM STATEMENT:

Sometimes it's better to use dynamic size arrays. Java's ArrayList can provide you this feature. Try to solve this problem using ArrayList.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of xth line.

Input Format

- The first line has an integer n.
- In each of the next n lines, there will be an integer d denoting the number of integers on that line, followed by d space-separated integers.
- In the next line, there will be an integer q denoting the number of queries.
- Each query will consist of two 1-based index integers: x (line number) and y (position in line).

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$
- Each number will fit in a signed 32-bit integer.
- The total number of integers across all lines will not exceed 10^5.

SOURCE CODE:

```
import java.io.*;

import java.util.*;


public class Solution {


   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();
List<List<Integer>> lines = new ArrayList<>();

for (int i = 0; i < n; i++) {
    int d = sc.nextInt();
    List<Integer> line = new ArrayList<>();
    for (int j = 0; j < d; j++) {
        line.add(sc.nextInt());
    }
    lines.add(line);
}

int q = sc.nextInt();

for (int i = 0; i < q; i++) {
    int x = sc.nextInt();
    int y = sc.nextInt();

    int lineIndex = x - 1;
    int elementIndex = y - 1;

    if (lineIndex >= 0 && lineIndex < lines.size() &&
        elementIndex >= 0 && elementIndex < lines.get(lineIndex).size()) {
        System.out.println(lines.get(lineIndex).get(elementIndex));
    } else {
        System.out.println("ERROR!");
    }
}
```

```
        sc.close();
    }
}
```

OUTPUT:

Input (stdin)

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

Your Output (stdout)

```
74
52
37
ERROR!
ERROR!
```

20.PROBLEM STATEMENT:

Given a list of non-negative integers nums, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: nums = [10,2]

Output: "210"

Example 2:

Input: nums = [3,30,34,5,9]

Output: "9534330"

Constraints:

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 10^9

SOURCE CODE:

```
class Solution {
    public String largestNumber(int[] nums) {
        String[] arr = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            arr[i] = String.valueOf(nums[i]);
        }
        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));
        if (arr[0].equals("0")) return "0";
        StringBuilder sb = new StringBuilder();
        for (String s : arr) sb.append(s);
        return sb.toString();
    }
}
```

OUTPUT:

Input

nums =

```
[10,2]
```

Output

```
"210"
```

Expected

```
"210"
```

21.PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

Given an array of Player objects, write a comparator that sorts them in order of **decreasing score**. If two or more players have the same score, sort those players **alphabetically by name**.

To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing Comparator.compare(T o1, T o2).

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

- The first line contains an integer, n, denoting the number of players.
- Each of the subsequent n lines contains a player's name and score, respectively.

Constraints

- $0 \leq$ score ≤ 1000
- Two or more players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

SOURCE CODE:

```
import java.util.*;

class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score == b.score) {
            return a.name.compareTo(b.name);
        }
```

```
        return Integer.compare(b.score, a.score);
    }
}

class Player {
    String name;
    int score;
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}
class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++) {
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++) {
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

OUTPUT:

Input (stdin)

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Your Output (stdout)

```
aleksa 150
amy 100
david 100
aakansha 75
```

22.PROBLEM STATEMENT:

You are given an array of strings names, and an array heights that consists of **distinct** positive integers. Both arrays are of length n.

For each index i, names[i] and heights[i] denote the name and height of the ith person.

Return names *sorted in **descending** order by the people's heights*.

Example 1:

Input: names = ["Mary","John","Emma"], heights = [180,165,170]

Output: ["Mary","Emma","John"]

Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: names = ["Alice","Bob","Bob"], heights = [155,185,150]

Output: ["Bob","Alice","Bob"]

Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

- n == names.length == heights.length
- 1 <= n <= 10^3
- 1 <= names[i].length <= 20
- 1 <= heights[i] <= 10^5
- names[i] consists of lower and upper case English letters.
- All the values of heights are distinct.

SOURCE CODE:

```
class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        Map<Integer, String> map = new HashMap<>();
        for (int i = 0; i < names.length; i++) {
            map.put(heights[i], names[i]);
        }
        Arrays.sort(heights);
```

```
        String[] result = new String[heights.length];
        int index = 0;
        for (int i = heights.length - 1; i >= 0; i--) {
            result[index] = map.get(heights[i]);
            index++;
        }
        return result;
    }
}
```

OUTPUT:

Input

names =

```
["Mary","John","Emma"]
```

heights =

```
[180,165,170]
```

Output

```
["Mary","Emma","John"]
```

Expected

```
["Mary","Emma","John"]
```